황보민

원스토어 클라이언트개발팀 기술 리드, 팀 AI 에이전트 하네스 구축, Android, iOS 앱 개발
010-4354-1115, 1stpie@naver.com

소개

원스토어 클라이언트개발팀의 기술 리드입니다. 3,500만 회원이 쓰는 원스토어 앱을 비롯해 뮤직메이트(FLO), 제주항공까지 Android, iOS에서 대고객 서비스를 만들어 왔습니다.

2026년 1월부터는 팀이 일하는 방식을 바꾸고 있습니다. 직접 설계한 팀 AI 에이전트 하네스에서는 Jira 일감 하나로 에이전트가 분석, 테스트, 구현, 리뷰, 검증을 거쳐 완료 보고까지 마치고, 사람은 지시하고 검수합니다. AI를 쓰지 않던 팀원까지 지금은 모두 이 환경에서 개발합니다.

기술 리드로서는 Clean Architecture 기반 구조를 팀 표준으로 정하고, 코드 리뷰 가이드와 팀 세션으로 팀원들이 같은 구조로 개발하게 이끌었습니다. 8년 넘은 국내 앱은 서비스를 멈추지 않은 채 계층별로 새 구조로 옮겼고, 글로벌 앱은 전체를 새 구조로 개발했습니다.

핵심 역량

역량	내용
팀 AI 에이전트 하네스 구축	**팀원 전원이 쓰는 개발 업무 자동화 AI 에이전트 하네스 설계, 구축, 운영 (2026.01~)** 역할별 에이전트로 컨텍스트를 나누고, 작업 에이전트를 병렬로 편성하는 멀티 에이전트 구조 설계 Jira 일감 하나로 요구사항 분석, 계획, 테스트, 구현, 리뷰, 실기기 검증, PR 생성과 리뷰까지 잇는 워크플로 자동화 SonarQube 이슈 수집과 Crashlytics 크래시 분석부터 수정, 빌드 검증, PR까지 잇는 품질 개선 자동화 데일리 코드 리뷰, 아키텍처 점검, PR 리뷰 반영 등 개발 워크플로 16종 제작 최대 16개 관점의 코드 리뷰와 실기기 화면, 로그, 서버 응답 검증을 통과할 때까지 반복하는 검증 루프 구축 팀원 작업 기록에서 반복 업무를 찾아 스킬, 훅, 에이전트로 자동화하고 업무 레퍼런스 작성 룰, 스킬, 훅, 에이전트 묶음을 플러그인 마켓플레이스로 팀에 배포하고, 사용 기록으로 매주 자가 개선 개인 산출물 지식베이스(LLM Wiki) 구축
기술 리드	**앱 구조 표준과 코드 리뷰, 기술 전파를 이끄는 클라이언트개발팀(9명) 기술 리드 (2023~)** Clean Architecture 기반 앱 구조와 계층별 모듈 구조, 개발 표준을 제안해 팀 합의로 확정 코드 리뷰 가이드를 팀 규칙으로 만들고, 리뷰어로 팀원 PR과 대규모 리팩토링 병합을 맡아 리뷰 문화 정착 샘플 아키텍처 프로젝트와 보통 2주마다 반복한 팀 세션으로 새 구조와 기술 전파 테스트 작성 기준을 팀 규칙으로 정하고, 일일 테스트 리포트와 실기기 UI E2E 자동화 구축(Jenkins, Maestro) 기술 부채 개선 과제를 정의하고 작업 계획, 팀원별 진행, 마감까지 관리 AK아이에스에서 항공 여정, 운임 조회 소규모 파트 리더로 고객사(제주항공) 요건 협의와 협력업체 코드 검수
리팩토링	**서비스를 멈추지 않고 8년 넘은 국내 앱의 전체 계층을 새 구조로 점진 전환 (2022.07~2025.09)** Clean Architecture 기반 멀티 모듈 구조로 Domain, Data, Presentation, DataSource, UI 순 전환 기능 개발과 병행해 레거시 코드와 새 구조를 함께 두고 Controller, 화면, 기능 단위로 하나씩 옮김 순환 참조와 UI 로직이 섞인 Controller를 풀어 Clean Architecture 의존성 규칙에 맞는 계층 구조로 정리 MVC를 MVVM, MVI로, XML 화면을 Compose로 전환하고 앱 시작 속도 개선
모바일 개발	**Android, iOS 두 플랫폼에서 대고객 서비스 앱 신규 개발과 대규모 개편** 3,500만 회원이 쓰는 원스토어 대고객 앱의 설치, 업데이트부터 전시, 회원, 결제 연동까지 서비스 핵심 흐름 개발 원스토어 글로벌 앱 개발과 브랜드별 앱 지원 모듈 구조, 설치 기능 구조 설계 뮤직메이트(FLO) 3.0 iOS 대고객 앱 고도화(Swift, RxSwift) 제주항공 대고객 앱 개발(Swift), 웹-앱 네이티브 브리지 개발, App Store 배포와 사내 앱 엔터프라이즈 배포
SDK 개발	**개발사와 빌드 서버가 쓰는 SDK, 라이브러리 설계와 개발** Google Play Core 라이브러리 역설계 분석을 바탕으로 원스토어 AAB Dynamic Delivery SDK 설계와 개발 빌드 서버가 쓰는 bundletool 라이브러리와 개발사 앱 APK, AAB 리패키징, 재서명 도구 개발 Box2D 물리엔진 기반 Android UI SDK 설계와 개발

경력

개발 경력 총 16년

원스토어 | 매니저, 정규직 | 2020.03 ~ 현재

원스토어 앱은 통신사 단말에 기본 탑재되는 앱마켓 앱입니다. 국내, 글로벌 앱의 대부분 영역을 개발해 왔고, 2023년부터 클라이언트개발팀의 기술 리드를 맡고 있습니다.

기간	프로젝트	담당
2026.01 ~ 현재	**팀 AI 에이전트 하네스 구축**	AI를 쓰지 않던 팀원까지 전원이 쓰는 팀 AI 에이전트 하네스 설계, 구축, 운영 일감 기준 워크플로, 역할별 에이전트, 훅 구성, 개인 산출물 지식베이스(LLM Wiki) 구축 플러그인으로 팀 배포, 회사 대상 구축 경험 발표와 팀 교육
2023.01 ~ 현재	**팀 기술 리드**	클라이언트개발팀(9명)의 기술 리드. 팀 개선 과제 설정과 관리, 앱 구조와 개발 표준 설계, 새 구조와 기술의 단계적 도입 방식 결정, 팀 전파와 학습 주도, 코드 리뷰 체계 도입
2023.12 ~ 현재	원스토어 글로벌 앱 고도화 개발	브랜드별로 앱을 지원할 수 있도록 모듈 구조 변경 설치 기능 리팩토링과 브랜드별 설치 기능 구조 설계(PackageInstaller) 다운로드, 업데이트, 화면 등 글로벌 앱 기능 고도화
2020.04 ~ 현재	원스토어 국내 앱 고도화 개발	다운로드와 설치, 자동 업데이트, 동적 전시와 상품 상세, 회원과 결제 연동 고도화 개발 (Kotlin, Foreground Service, Data Binding, Coroutine)
2026.07 ~ 2026.10	웹게임 플랫폼 개발과 광고 플랫폼 연동	팀이 AI 하네스로 만든 HTML5 웹게임 플랫폼의 웹 SDK, 개발사 테스트 빌드 실행 경로, 마이크와 카메라 권한 처리 개발(Kotlin, JavaScript, WebView) 공용 광고 모듈 설계와 국내, 글로벌 앱 적용, 광고 브리지와 ADX 연동, 서명 기반 광고 규격 전환(Android, Kotlin)
2026.03 ~ 2026.04	단위 테스트와 테스트 자동화 구축	테스트 작성 체계 설계, 작성 가이드 스킬로 여러 에이전트가 나눠 단위 테스트 작성(Kotest, MockK) 계측 테스트 복구(Android), 일일 테스트 리포트와 실기기 UI E2E 자동화(Jenkins, Maestro)
2025.06 ~ 2026.01	**전사용 디자인 시스템 구축**	전사 앱이 함께 쓰는 디자인 시스템의 기술 리서치와 구조 설계(Gradle, Maven) Figma 값을 Kotlin 디자인 토큰 코드로 바꾸는 파서와 공통 Compose 컴포넌트 개발 (Kotlin, Figma, Jetpack Compose)
2023.04 ~ 2023.10	원스토어 글로벌 앱 개발	브랜드별로 나뉘는 글로벌 앱의 멀티 모듈 구조 설계(Clean Architecture, 모듈러 아키텍처) MVVM 구조로 다운로드와 설치 기능 개발(Hilt, Jetpack Compose, Flow)
2022.07 ~ 2025.09	**원스토어 국내 앱 점진적 리팩토링**	서비스 중인 국내 앱을 Domain, Data, Presentation, DataSource, UI 순으로 전체 계층 리팩토링 Clean Architecture 기반 멀티 모듈 구조 설계와 일정 관리, MVC를 MVVM, MVI로 전환 XML 화면 Compose 전환, 다운로드와 설치, 회원 연동 기능 재구성, 앱 시작 속도 개선
2021.09 ~ 2022.03	스티커스토어: 삼성 단말 메시지 앱 확장 기능 개발	스티커팩과 이모지의 구매, 다운로드, 편집 화면 개발(Kotlin, RxJava, Retrofit, MVVM, 단방향 데이터 흐름)
2020.12 ~ 2024.10	지원 도구 개발	개발사 앱 리패키징과 재서명 도구, AAB 단말 설치 도구, 앱 검증 팀 지원 도구 개발(Kotlin, Java, Python, bundletool, apksig, adb)
2020.10 ~ 2021.10	**원스토어 AAB SDK, 라이브러리 개발**	Google Play Core 라이브러리 역설계 분석과 원스토어용 AAB Dynamic Delivery SDK 설계, 개발(Android App Bundle, Play Asset Delivery) SDK와 원스토어 앱 간 연동 개발(Android, AIDL) 원스토어용 bundletool 라이브러리 개발(bundletool)

AI 기술

분야	기술
AI 도구	Claude Code CLI, Cursor
에이전트 아키텍처 패턴	오케스트레이터-워커, 계층형 멀티 에이전트, 모델 라우팅
검증, 평가	품질 게이트, 작업자와 분리된 LLM 검증 에이전트, 완료 기준 채점과 판정 스크립트
에이전트 워크플로	평가-개선 루프, 서킷 브레이커, 단계별 상태 저장과 재개
컨텍스트 엔지니어링	상시 룰과 경로별 조건부 룰 분리, 역할별 에이전트로 컨텍스트를 나누고 에이전트별 도구 권한 제한
Agent Skills	워크플로 스킬, 테스트 작성 가이드 스킬, 주간보고 스킬 등 팀 스킬 직접 제작
MCP	스크립트 기반 MCP 도구 구현: SonarQube 조회, 지식베이스 검색
하네스 엔지니어링	훅 기반 가드레일, 사내 플러그인 마켓플레이스 배포, 사용 기록 기반 주간 개선, 개인 메모리의 팀 하네스 편입
지식 관리	LLM Wiki 방식의 로컬 Markdown 산출물 지식베이스
언어	Python

AK아이에스 | 항공서비스팀 과장, 정규직 | 2018.07 ~ 2020.03

제주항공 대고객 iOS 앱과 웹 프론트엔드, 백엔드 개발을 담당했고, 2019년 6월부터 항공 여정, 운임 조회 소규모 파트 리더를 맡았습니다.

기간	프로젝트	담당
2019.06 ~ 2020.03	항공 여정, 운임 조회 소규모 파트 리더	여정, 운임 조회 웹과 앱 개발 리드, 고객사 요건 협의와 정리, 협력업체 개발 코드 리뷰와 검수
2018.07 ~ 2020.03	제주항공 앱 개발	대고객 iOS 앱의 회원, 모바일 탑승권, 웹-앱 네이티브 브리지 개발과 App Store 배포 (Swift, Objective-C, RxSwift, Alamofire, Kingfisher, SQLite) 정비사, 승무원, 기장, 중국 POS용 사내 앱 고도화와 엔터프라이즈 배포
2018.07 ~ 2020.03	제주항공 웹 프론트엔드와 백엔드 개발	제주항공 모바일 웹의 여정, 운임 조회 화면과 안내 페이지 고도화, 백엔드 개발(jQuery, Spring 3.1) 제주항공 서비스 콘텐츠 관리 시스템 개발(AngularJS 1.7, Spring Boot 2)

미래ICT | 프리랜서 | 2018.03 ~ 2018.06

기간	프로젝트	담당
2018.03 ~ 2018.06	현대모비스 차세대 내비게이터 선행 개발	차량 MCU 연동 HIDL HAL 포팅(AOSP, C++)

이노에이스 | 모바일서비스 개발3팀 과장, 정규직 | 2010.01 ~ 2017.10

기간	프로젝트	담당
2017.01 ~ 2017.07	뮤직메이트(FLO) 3.0 개발	뮤직메이트(FLO) 3.0 iOS 대고객 앱 고도화 개발(Swift, RxSwift, Alamofire, Moya, Realm)
2014.11 ~ 2017.01	티스토어, 원스토어 앱 개발	티스토어 신규 개발과 고도화, 원스토어 6.0 UI 전면 개편(Android, Java) 전시, 상세, 검색, 선물, 구매 화면의 UI 레이어와 비즈니스 로직, 서버 연동 개발(Glide, Volley, OkHttp) 스토어 설치, 인증, 회원 가입을 돕는 선탑재 보조 앱 개발
2014.07 ~ 2014.10	SK Physics Android SDK	물리 스레드 기반 View 갱신 구조 설계, Android View와 Box2D 물리엔진을 연동하는 SDK 개발(Android, Java, Box2D, Animator)
2014.04 ~ 2014.05	SKT AR브라우저	기존 AR 그래픽 엔진에 QR 인식 시 동영상이 재생되도록 하는 기능 개발(Android, C++, JNI, FFmpeg)

기간	프로젝트	담당
2013.04 ~ 2014.03	SKT 기본 탑재 런처 T간편모드	런처의 Android 영역 개발: 2G폰 키패드형 홈 UI, 홈 모드 전환, 시스템 권한 단말 설정 제어, 3사 단말 포팅(Android, Java, Hidden API)
2011.07 ~ 2013.06	HTML5 웹앱 개발 도구 다빈치	위젯 컴포넌트와 이벤트 래핑을 담은 jQuery 기반 웹앱 프레임워크 개발(JavaScript, jQuery, HTML5, CSS3) 웹앱을 Android 앱으로 만드는 하이브리드 빌드와 단말 연동, 샘플 앱과 템플릿 제작
2011.02 ~ 2011.05	증권 MTS MP트래블러 II 홈 런처	증권 MTS의 홈 런처와 위젯 개발(Java, C++, Lua, OpenGL ES)
2010.08 ~ 2011.06	Android 런처 앱과 테마 상점 개발	폰꾸미기(SKT 선탑재, MUIF 엔진 기반, 400만 이상 다운로드)와 Rabbit Launcher, SmaDeco(글로벌 판매)의 런처 화면, 엔진 렌더링 연동, 테마 상점 개발(Java, C++, Lua, OpenGL ES, HttpClient)
2010.01 ~ 2011.06	3D UI 프레임워크 MUIF	엔진 디버깅, SDK 컴포넌트, WIPI-C 샘플 앱, 저작도구 UI 컴포넌트 개발(C, C++, Lua, OpenGL, WIPI-C)
2009.09 ~ 2009.12	앱마켓 플랫폼 사업 (인턴, 입사 전)	WIPI-C, Windows Mobile, Android 개발(C, C++)

학력, 자격

- **학력:** 단국대학교 컴퓨터과학과 학사(심화전공, 경영학 복수전공) (2003.02 ~ 2010.02)
- **자격:** 정보처리기사 (2009)

사용 기술

협업, 도구

분야	기술
형상 관리	Git, Bitbucket, SVN
이슈 관리	Jira, Mantis, Redmine
협업	Confluence, Teams
CI, 정적 분석	Jenkins, SonarQube

자기소개서

황보민, 쿠팡이츠 Staff Mobile Engineer(배달파트너, 스토어 앱) 지원
010-4354-1115, 1stpie@naver.com

원스토어 클라이언트개발팀의 기술 리드입니다. 3,500만 회원이 쓰는 원스토어 Android 앱을 개발하면서 팀의 앱 구조와 개발 표준, 코드 리뷰를 이끌고 있습니다.

[지원 동기]

Android를 깊게 다뤄 왔고, 9명의 Android 팀을 이끌고 있습니다. 앱마켓의 설치와 업데이트처럼 플랫폼에 맞닿은 기능부터 전시, 회원, 결제 같은 커머스 흐름, 글로벌 앱과 SDK까지 여러 영역을 맡아 왔습니다. 그 가운데 8년 넘은 국내 앱을 서비스를 멈추지 않고 새 구조로 바꾼 일처럼, 레거시를 개선하는 일을 즐깁니다. 공고에서 쿠팡이츠가 Flutter 기술 스택을 여러 기술과 비즈니스 도메인으로 넓혀 가는 일을 보고, 네이티브에서 쌓은 플랫폼 이해와 구조를 바꿔 온 경험으로 그 전환을 함께하려고 지원했습니다. Flutter는 2022년 동료와 둘이서 한 팀 내부 과제에서 원스토어 앱의 기본 화면을 Flutter로 만들고, 다운로드와 설치는 Android 네이티브 코드와 연동해 봤습니다.

[공고 업무와 맞는 경험]

- **확장 가능한 앱 아키텍처:** Clean Architecture 기반 앱 구조와 계층별 모듈 구조를 제안해 팀 합의로 표준을 정했습니다. 2023년에는 브랜드별로 나뉘는 원스토어 글로벌 앱의 멀티 모듈 구조를 설계해 개발했고, 이어서 브랜드별로 앱을 지원할 수 있도록 모듈 구조를 바꾸고, 설치 기능을 리팩토링하며 브랜드별 설치 기능 구조를 설계했습니다.
- **복합적인 기술 이슈와 리팩토링:** 8년 넘은 국내 앱은 Controller끼리, Controller와 View가 서로 참조하고, 데이터 로드를 맡은 Controller 11개에 UI 로직까지 섞여 있었습니다. 서비스를 멈출 수 없어 기능 개발과 병행하며 Domain, Data, Presentation, DataSource, UI 순으로 전체 계층을 옮겼습니다. 순환 참조는 인터페이스와 의존성 주입으로 끊고, Controller는 하나씩 Repository와 UseCase로 옮겼습니다. 아직 Java로 남은 코드도 새 Flow 기반 코드를 쓸 수 있게 확장 함수를 두어, 레거시 코드와 새 구조가 함께 쓰이게 했습니다.
- **코드 리뷰와 기술 전파:** 리뷰 요청 작성, 리뷰 속도, 주요 관점, 코멘트 작성법을 담은 코드 리뷰 가이드를 정리해 팀 규칙으로 만들고, 리뷰어로 팀원 PR과 대규모 리팩토링 병합을 맡았습니다. 새 구조는 샘플 아키텍처 프로젝트로 먼저 보여 주고, 아키텍처, Coroutine, Flow, Hilt, 테스트를 주제로 한 팀 세션을 보통 2주마다 이어 갔습니다.
- **네이티브 플랫폼:** Android에서는 PackageInstaller 세션 설치, 다운로드와 자동 업데이트의 백그라운드 작업, AIDL로 원스토어 앱과 연동하는 AAB SDK를 개발했고, 새 Android 버전이 나오기 전에 설치 세션과 알림 권한 변화에 대응했습니다. iOS에서는 제주항공 대고객 앱을 Swift로 개발해 App Store로 배포하고, 뮤직메이트(FLO) 3.0 iOS 앱을 고도화했습니다.
- **커머스 서비스:** 원스토어 앱에서 홈, 패널, 카드 동적 전시와 앱, 게임, 쇼핑 상세, 구매와 결제 내역, 회원과 결제 연동을 맡아 왔고, 로그인 구조 재구성과 토큰 만료 시 자동 연장을 개발했습니다.
- **CI와 개발 프로세스 자동화:** 기능 개발 때 테스트를 함께 쓰는 규칙을 팀에 적용하고, 매일 단위, 계측 테스트를 돌려 결과를 공유하는 리포트(Jenkins)와 실기기 UI E2E 자동화(Maestro)를 구축했습니다. 2026년 1월부터는 팀 AI 에이전트 하네스를 설계, 구축했습니다. Jira 일감 하나로 에이전트가 분석, 테스트, 구현, 리뷰, 검증을 거쳐 PR과 완료 보고까지 이어 가고, 사람은 지시하고 검수합니다. AI를 쓰지 않던 팀원까지 지금은 모두 이 환경에서 개발합니다.

[입사하면 먼저 맡을 일]

배달파트너와 스토어 앱의 신규 기능을 개발하면서, 모듈 경계와 의존 방향, 코드 리뷰 기준을 팀과 함께 다듬는 일입니다. 계층을 나누고 의존 방향을 지키는 기준, 서비스를 멈추지 않고 구조를 바꾸는 방식, 네이티브 연동 경험을 Flutter 앱에도 그대로 가져갈 수 있습니다.

경력 상세와 구조 그림은 첨부한 이력서와 포트폴리오에 담았습니다.

1. 팀 AI 에이전트 하네스 구축

원스토어 클라이언트개발팀, 2026.01 ~ 현재, 설계, 구축, 운영. AI를 쓰지 않던 팀원까지 전원이 이 하네스로 개발하고, 도입 뒤 QA 검증 기간 결함은 본인 담당분 0건, 팀 전체로는 이전 대비 절반 이상 줄어듦

하네스 구성

팀원 전원이 쓰는 하나의 하네스

`rules (11)`	always: 팀 규칙, 팀 관례 paths: 아키텍처 계층별 룰, 코드 주석 규약, md 작성 규약
`references`	리뷰 기준, Git, Confluence, 스킬 실행 규약, Jira 흐름
`memory`	개인 메모리에서 승격한 팀 기억, 개인 금지 룰
`agents (19)`	계획, 아키텍처 설계, 구현, 레거시 전환, UX, 테스트, 코드 리뷰, 문서 리뷰, 보안 리뷰, 검증, 판단 보조, 코드 탐색, 문서 수집, 코드 최적화, SonarQube 수정, 형상 관리, 작업 관리, 팀 조율(2)
`procedures`	에이전트별 절차, XML→Compose 파이프라인, 정적 분석, 리뷰 축, 실기기 테스트, 하네스 개선
`skills (45)`	기능 구현 워크플로, 크래시 수정 워크플로, PR 생성 리뷰 워크플로, PR 요청, 코드 리뷰, 심층 분석, 정리, 형상 관리, 하네스 개선, 일감 시작, 하위 일감 생성, 주간보고, Figma 디자인, 하네스 점검, 하네스 동기화 등
`hooks`	보호 브랜치 차단, 위험 명령 차단, 외부 전송 차단, 수정 후 동작, push 게이트 등
`scripts`	빌드 도구, 실기기 테스트 도구, 사용 통계 전송 도구
`MCP, LSP`	mobile-mcp, SonarQube 조회, Kotlin LSP
`프로젝트 정보`	앱 저장소마다 형상 관리: 프로젝트 기본 정보, 규칙, 아키텍처, 레거시 정보, 문서

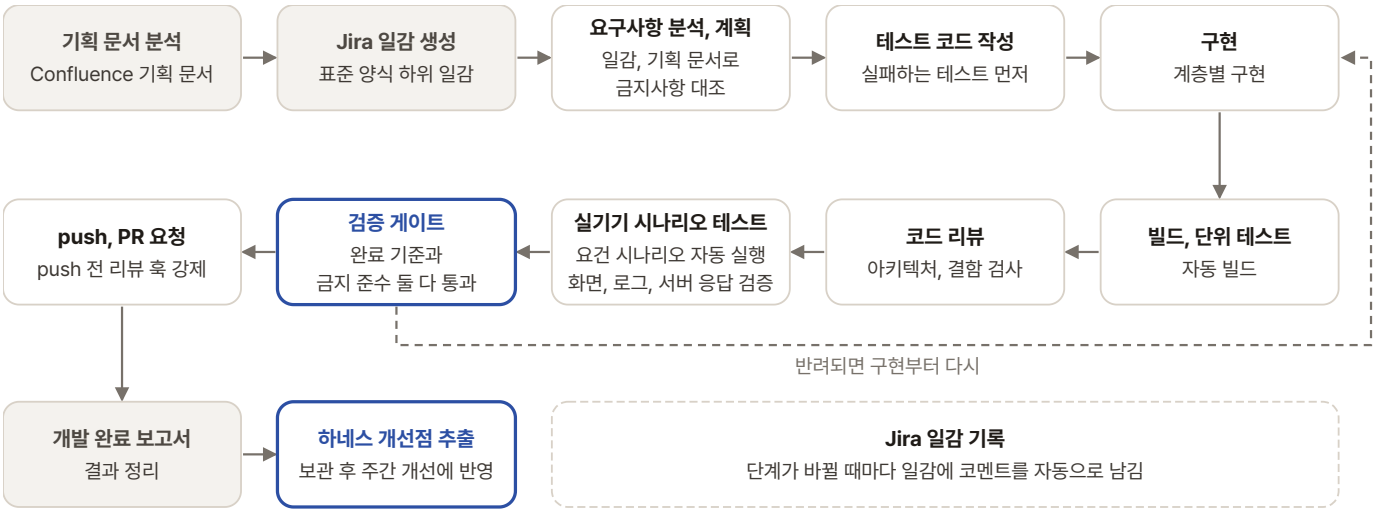
특징

- 플러그인으로 팀에 배포해 팀원 전원이 같은 환경에서 일하고, 팀원의 업무가 AI 자산이 됨
- 앱별 하네스(프로젝트 정보, 규칙, 아키텍처, 레거시 정보, 문서)는 각 앱 저장소에 두어 코드와 함께 형상 관리
- 팀원의 금지 결정을 저장해 무인 세션에 반영
- 주마다 팀원의 사용 데이터를 바탕으로 하네스를 개선해 팀 AI 자산화
- 워크플로 스킬로 일감 하나에서 업무를 시작하고, 단계마다 상태를 보고하고 저장해 진행이 끊겨도 그 단계부터 재개
- 역할별 에이전트를 통한 컨텍스트 격리
- 팀원 작업 기록에서 반복 업무를 찾아 스킬로 만듦
- Andrej Karpathy의 LLM Wiki 기반 지식베이스를 통한 개인 산출물 관리

팀 전원이 쓰게 만든 배포

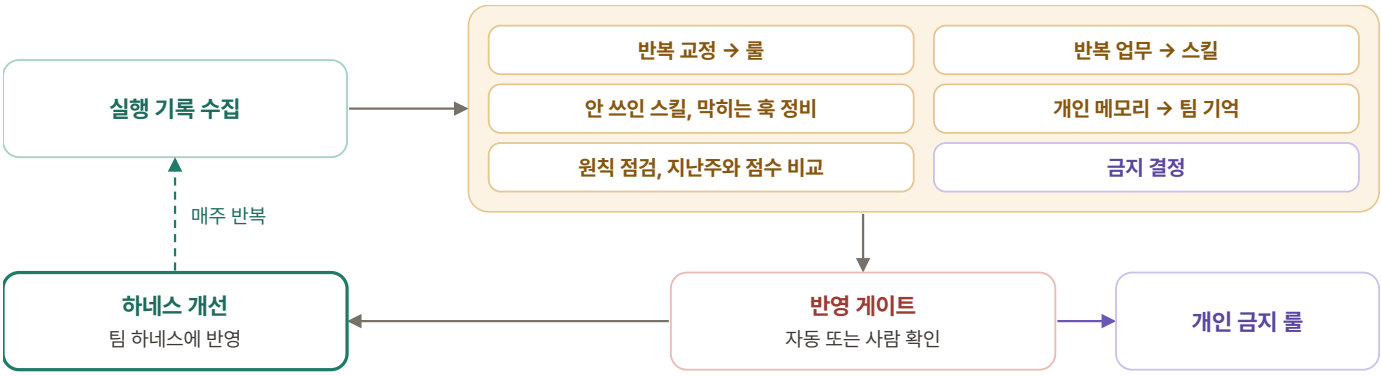
- 사내 플러그인 마켓플레이스에 하네스를 올림. 처음에 사용자 정보와 작업하는 앱 저장소만 입력하면 동기화 스킬이 설치와 갱신을 맡고, 하네스 변경을 머지하면 커밋 SHA 기준으로 배포
- 등록된 앱 저장소마다 앱 규칙, 권한, 검증 장치를 점검해 빠진 것만 확인을 받고 채움. 세션을 시작할 때 하루 한 번 연결과 설정을 점검해 깨진 것을 고침
- 팀원들의 스킬, 훅 사용 기록으로 하네스 개선 및 업무 스킬화

일감에서 PR까지 하나의 흐름



- **팀 개발 프로세스를 에이전트 루프로 설계:** 일감에서 PR, 완료 보고까지 하나의 흐름으로 만들어, 사람은 지시, 감독, 검수를 맡는 방식으로 팀 개발을 바꿈
- **속도보다 품질을 먼저 지키는 구조:** 테스트를 먼저 쓰고, 완료는 검증 게이트가 판정
- **사람과 AI의 경계 설계:** 계획과 최종 소스는 사람이 보고, 금지 규칙을 어겨야 하는 상황이면 에이전트가 멈추고 물음
- **관리 비용을 줄이는 투명성:** 진행 상황이 Jira에 자동으로 남아 PM이 에이전트 기록으로 파악

매주 점검하고 고치는 하네스



- **쓸수록 좋아지는 하네스:** 반복 교정과 반복 업무를 룰과 스킬로 승격해, 팀원의 경험이 팀 자산으로 쌓임
- **데이터로 운영:** 원칙 점검 점수를 주마다 비교하고, 안 쓰인 스킬과 막히는 훅을 사용 기록으로 찾아 정비
- **팀 표준과 개인 판단의 분리:** 팀 하네스 변경은 반영 게이트로 통제하고, 개인 금지 결정은 개인 룰로 따로 관리

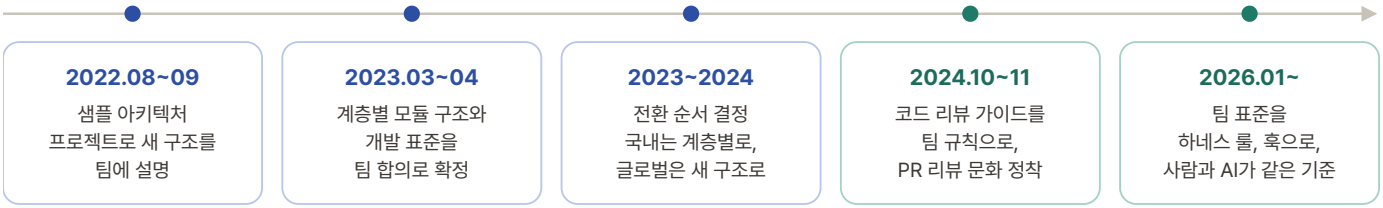
그 밖의 실적

- 코드 리뷰: 최대 16개 관점 리뷰, Kotlin LSP로 호출 관계 추적, 데일리 리뷰 결과를 Confluence KPI 대시보드로 공유
- 빌드, 테스트 자동 검증: 수정 파일마다 ktlint, Kotlin LSP 검사, 빌드 대기열, 실기기 E2E 스크립트 판정, push 전 아키텍처 검사
- SonarQube 조회를 스크립트 기반 MCP 도구로 구현하고, 팀 사용 통계 수집 서버 개발
- Crashlytics 원인 분석부터 승인 뒤 수정과 PR까지 자동화
- 팀 교육용 Claude Code 에이전트 가이드 12편과 기초 가이드 2편을 사내 위키에 쓰고, 회사 대상 AI 발표로 구축 경험을 공유

2. 기술 리드

새 구조를 정하고, 팀이 익혀 쓰게 만들기

원스토어, 2023.01 ~ 현재, 클라이언트개발팀(9명) 기술 리드, 아키텍처 개선 주도는 2022.07부터



기술 부채 과제 관리 (2023~)
국내 앱 리팩토링 등을 과제로 정의하고
작업 계획, 팀원별 진행, 릴리스 마감까지 관리

- **아키텍처 가이드:** 앱 구조와 개발 표준을 샘플 아키텍처 프로젝트로 먼저 보여 주고 팀 합의로 확정. 도입 방식도 정해 국내 앱은 계층별로 옮기고, 글로벌 앱은 새 구조로 개발
- **코드 리뷰:** 리뷰 요청 작성, 리뷰 속도, 주요 관점, 코멘트 작성법과 처리를 담은 코드 리뷰 가이드를 정리해 팀 규칙으로 옮기고, 리뷰어로 팀원 PR을 검토하고 대규모 리팩토링 병합을 맡음
- **기술 전파:** 아키텍처(MVVM, MVI, Clean Architecture, 모듈화), 비동기와 의존성 주입(Coroutine, Flow, Hilt), 테스트(테스트 코드, TDD)를 주제로 팀 세션을 보통 2주마다 반복
- **코드 품질과 개발 프로세스:** 계층별 테스트 작성 기준을 정해 기능 개발 때 테스트를 함께 쓰는 규칙을 팀에 적용하고, 매일 단위, 계측 테스트를 돌려 결과를 공유하는 리포트와 실기기 UI E2E 자동화 구축(Jenkins, Maestro)
- **사람과 AI가 같은 기준:** 계층 규칙과 리뷰 기준을 하네스 룰로 옮기고, 훅과 아키텍처 검사로 지키게 함
- **결과:** 국내, 글로벌 두 앱이 같은 구조 표준을 따르게 됐고, PR 리뷰가 팀 규칙으로 자리 잡음

항공 여정, 운임 조회 소규모 파트 리더

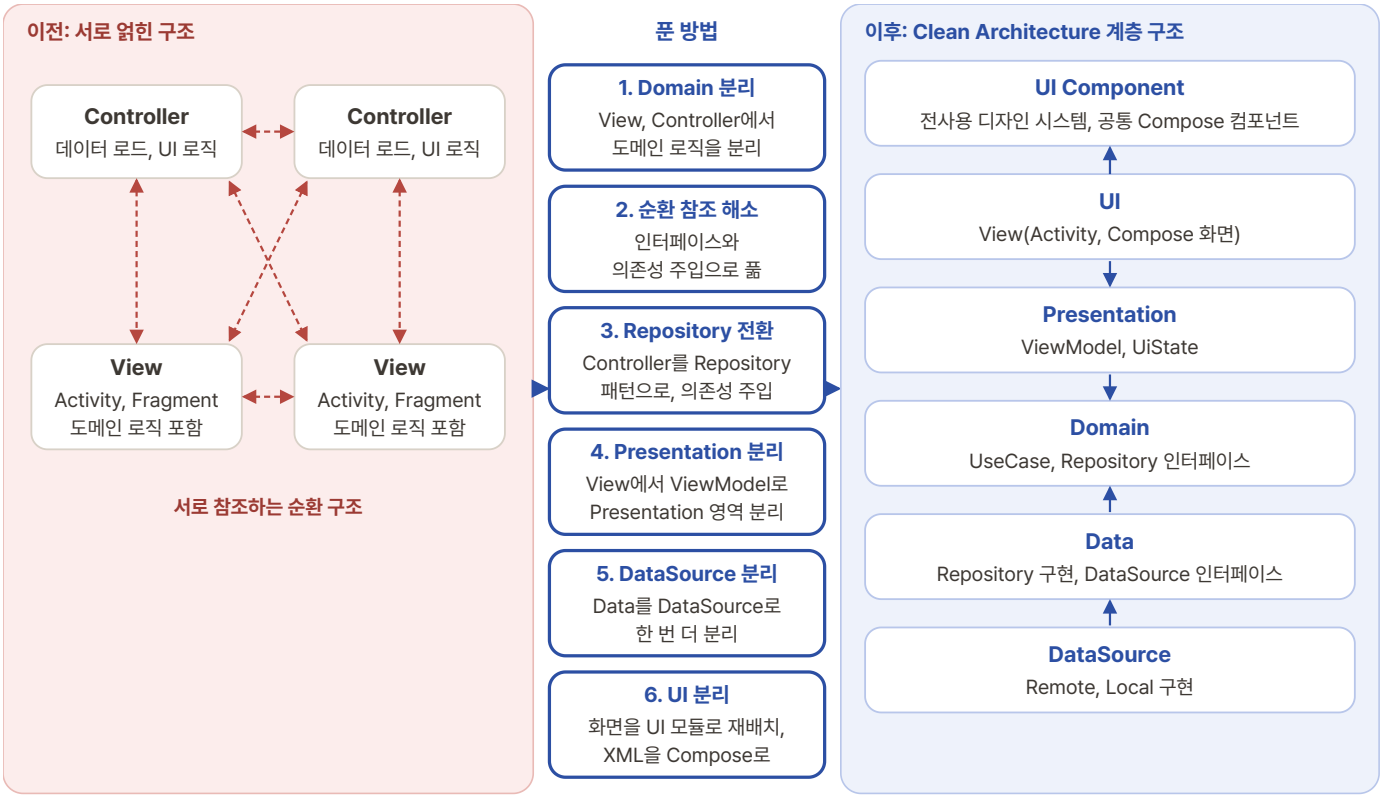
AK아이에스 (제주항공), 2019.06 ~ 2020.03

- **요건과 품질을 함께:** 여정, 운임 조회 웹과 앱 개발을 이끌며 고객사(제주항공) 요건을 직접 협의, 정리하고, 협력업체가 맡은 대고객, 사내 웹과 모바일 앱의 코드를 리뷰, 검수

3. 리팩토링

서비스 중인 원스토어 국내 앱의 전체 계층 리팩토링

원스토어, 2022.07 ~ 2025.09, 기능 개발과 병행해 Domain, Data, Presentation, DataSource, UI 순으로 전환



- **배경:** 8년 넘은 국내 앱에 MVC, 일부 MVP 코드와 기술 부채가 쌓였고, 서비스를 멈추지 않고 구조를 바꿔야 했음
- **맡은 범위:** 국내 앱 리팩토링은 여러 명이 함께 한 팀 과제였고, 본인은 이를 이끌며 구조 설계와 일정 관리, 대규모 리팩토링 병합을 맡고, Domain 분리, 순환 참조 해소, Data 분리, UI 작업, DataSource 분리 등 계층 전환 대부분을 직접 함
- **이후 구조:** UI는 Presentation에, Presentation과 Data는 Domain에, DataSource 구현은 Data의 인터페이스에 의존하는 Clean Architecture 구조. 리팩토링 뒤 전사용 디자인 시스템을 추가로 적용해 공통 Compose 컴포넌트와 함께 UI Component 계층으로 둠

구조를 잡은 방법

- **검토와 시범:** 2022.07 코루틴 도입 가능성과 선언형 UI, 그에 맞는 아키텍처를 검토하고, 샘플 아키텍처 프로젝트로 새 구조를 팀에 먼저 설명
- **계층과 의존 방향:** Domain이 Repository 인터페이스를, Data가 DataSource 인터페이스를 갖게 해 두 경계에서 의존 방향을 뒤집고, 계층마다 전용 모델과 Mapper를 둠
- **오류 처리 기준:** 예상된 실패는 Result로, 예상 밖 실패는 예외로 다루고, Data 계층에서 프레임워크 예외를 Domain 오류로 바꿈
- **모듈:** domain 모듈을 새로 만들고, 계층별 모듈 구조를 채택해 새 구조의 뼈대를 만듦

레거시와 새 구조의 공존

- **기능 개발과 병행:** 한 번에 바꾸지 않고, 레거시 코드와 새 구조를 함께 두고 Controller, 화면, 기능 단위로 하나씩 옮김
- **레거시 코드에서 새 코드 쓰기:** 콜백 기반 로더를 Flow 기반으로 바꾸면서, 아직 Java로 남은 코드도 Flow를 쓸 수 있게 확장 함수와 도메인 예외 타입을 추가
- **주입 범위 넓히기:** Hilt와 data 모듈을 추가하고 적용할 수 있는 곳부터 싱글턴 직접 참조를 Hilt 주입으로 바꿈
- **하나씩 옮기고 확인 뒤 병합:** Java 코드를 Kotlin으로 바꾼 리팩토링 브랜치는 한 화면을 먼저 전환해 놓친 예외 상황을 확인한 뒤 병합
- **서비스에서 Worker로:** 핵심 다운로드, 설치, 자동 업데이트 경로를 맡던 서비스의 기능을 하나씩 Worker와 UseCase로 옮기고, 남은 기능까지 옮긴 뒤 서비스를 제거

진행 순서

- **Domain:** View와 Controller에 흩어진 도메인 로직을 떼어 domain 모듈로 모음
- **Data:** Controller를 Repository 패턴으로 바꾸고, 싱글턴 직접 참조를 Hilt 주입으로, AsyncTask와 콜백 기반 로더를 Coroutine, Flow로, Java 코드를 Kotlin으로 바꿈
- **Presentation:** View에서 ViewModel로 Presentation 영역을 분리하고, MVC를 MVVM, MVI로 전환
- **DataSource:** Data를 DataSource로 한 번 더 분리해 Remote, Local 구현이 Data의 인터페이스에 의존하게 함
- **UI:** 화면을 UI 컴포넌트와 View로 나눠 UI 모듈로 재배치하고, XML 화면을 Compose로 전환
- **기능:** 다운로드와 설치, 회원 연동 기능을 새 구조에 맞춰 재구성
- **결과:** 서비스 중인 국내 앱의 전체 계층을 새 구조로 옮김

어려웠던 점: 얽힌 Controller와 순환 참조

- **문제:** Controller끼리, Controller와 View가 서로 참조하고, 도메인 로직이 View와 Controller에 흩어져 있었음. 데이터 로드를 맡은 Controller 11개에는 UI 로직까지 섞여 있었음
- **푼 방법:** 순환 참조는 인터페이스와 의존성 주입으로 끊고, domain 모듈로 옮기는 작업을 마친 뒤 남은 순환 참조를 고침. Controller는 하나씩 Repository와 UseCase로 옮김
- **결과:** 순환 참조와 UI 로직이 섞인 Controller를 풀어 Clean Architecture 의존성 규칙에 맞는 계층 구조로 정리

4. 모바일 개발

Android, iOS 앱 개발

이노에이스, AK아이에스, 원스토어, 2010.01 ~ 현재

2010 ~ 2014

런처와 UI 엔진

폰꾸미기
(선탑재, 400만 이상 다운로드)
T간편모드(기본 탑재 런처)
Rabbit Launcher, MUIF 3D UI

2014 ~ 2017

앱마켓

티스토어 신규 개발, 고도화
원스토어 6.0 UI 전면 개편
전시, 상세, 검색, 선물, 구매

2017 ~ 2020

iOS 앱

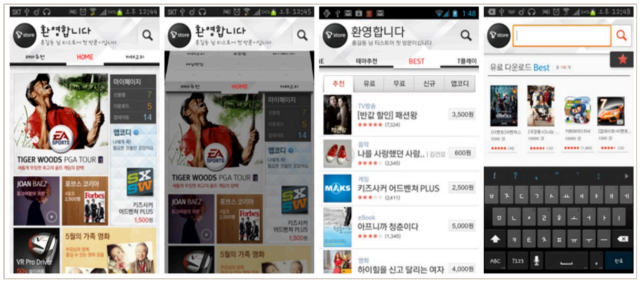
뮤직메이트(FLO) 3.0
제주항공 대고객 앱, 사내 앱
모바일 탑승권

2020 ~ 현재

원스토어 국내, 글로벌

3,500만 회원 앱마켓
다운로드, 설치, 업데이트
전시, 회원, 결제, SDK

- **3,500만 회원 앱마켓:** 통신사 단말에 기본 탑재되는 원스토어 국내 앱의 다운로드, 설치, 자동 업데이트, 동적 전시와 상품 상세, 회원과 결제 연동을 담당하고, 2023년 글로벌 앱을 멀티 모듈 구조로 설계해 개발하고, 이후 브랜드별로 앱을 지원하도록 모듈 구조를 바꾸고 브랜드별 설치 기능 구조를 설계
- **플랫폼 깊이:** 시스템 권한으로 단말 설정을 제어하는 기본 탑재 런처(T간편모드), PackageInstaller 세션 설치, AIDL로 다른 앱과 연동하는 SDK를 개발하고, 새 Android 버전 출시 전에 선제 대응
- **iOS:** 뮤직메이트(FLO) 3.0 iOS 대고객 앱 고도화 개발(Swift, RxSwift). 제주항공 대고객 앱을 Swift, Objective-C로 개발하며 웹-앱 네이티브 브리지를 구현해 App Store로 배포하고, 정비사, 승무원, 기장, 중국 POS용 사내 앱은 고도화해 엔터프라이즈 인증서로 배포



티스토어: 홈, 베스트 목록, 검색 화면



T간편모드: 홈 모드 전환, 키패드형 홈, 화면, 소리 설정, 간편 메뉴

전사용 디자인 시스템 구축

원스토어, 2025.06 ~ 2026.01

Figma 디자인에서 각 앱까지, 전사 앱이 함께 쓰는 흐름

Figma 디자인
디자인 원본

→

토큰 파서
Figma 값을
Kotlin 코드로

→

디자인 토큰
색, 글꼴,
간격, 모서리

→

공통 컴포넌트
Compose
Button, Card 등

→

AAR 빌드
사내 Maven
배포

→

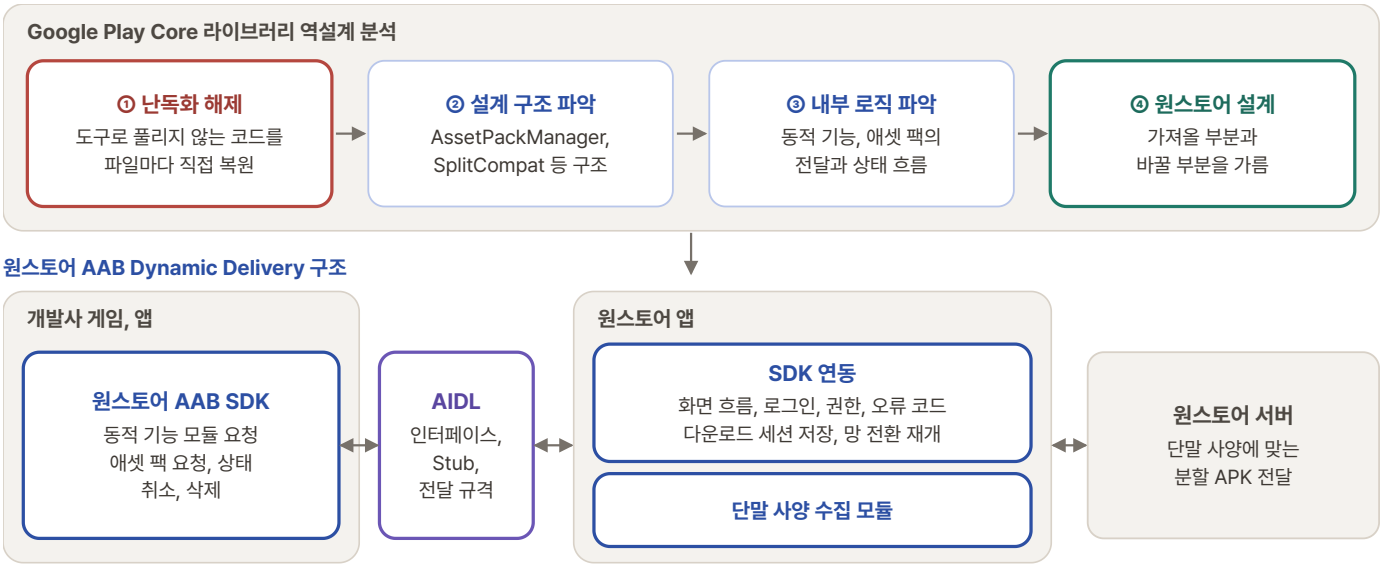
각 앱
버전 지정해
의존성으로 사용

- **맡은 일:** 전사용 디자인 시스템 구축 TF에서 기술 리서치와 구조 설계를 하고, Figma 값을 Kotlin 디자인 토큰 코드로 바꾸는 파서와 공통 Compose 컴포넌트를 개발

5. SDK 개발

AAB Dynamic Delivery SDK: Google Play Core 라이브러리 역설계 분석에서 설계까지

원스토어, 2020.10 ~ 2021.10



- **문제:** 원스토어는 AAB를 지원하지 않아 개발사 앱이 동적 기능 모듈, 애셋 팩을 원스토어에서 내려받을 방법이 없었음
- **가장 어려웠던 일:** 참고할 Google Play Core 라이브러리는 난독화돼 있고 일부 클래스는 디컴파일도 되지 않아 도구로 한 번에 풀 수 없었음. 파일을 하나씩 직접 분석해 난독화를 풀며 복원함
- **설계, 구현:** SDK와 원스토어 앱 사이 AIDL 규격을 설계하고, 원스토어 앱 쪽 SDK 연동과 단말 사양 수집 모듈을 구현. 개발사가 참고할 샘플 앱과 SDK 개발 가이드 작성

그 밖의 SDK, 라이브러리, 도구

- **원스토어용 bundletool 라이브러리:** 원스토어 앱 빌드 서버가 불러 쓰는 JAR 라이브러리로 개발
- **APK와 AAB 리패키징, 재서명 도구:** 글로벌 스토어 개발사 앱을 다시 패키징, 서명하는 Kotlin 실행 JAR. bundletool, apksig로 서명 체계 (v1~v4) 검증, 여러 서명 키 처리
- **SK Physics Android SDK (이노에이스, 2014.07 ~ 2014.10):** 물리 스레드 기반 View 갱신 구조를 설계해 Android View와 Box2D 물리엔진을 연동